Mini C Compiler

Course: CS-471L Compiler Construction

Team Members

Muhammad Hassnain	2022-CS-670
Abdullah Ateeq	2022-CS-661
Tayyab Zahid	2022-CS-659

Lab Supervisor

Madam Ghazala

Department of Computer Science, New Campus

University of Engineering and Technology Lahore, Pakistan

1. Introduction

This project implements a **Mini-C-Compiler** for a restricted subset of a C-like programming language. The objective is to demonstrate the integration of major compiler phases and simulate the functioning of real-world system compilers.

The implemented compiler processes source code through the following phases:

Lexical Analysis (Scanner)-> Syntax Analysis (Parser)-> Semantic Analysis-> Intermediate Code Generation

2. Problem Statement

Design and Implementation of a **Mini-C-Compiler** for a Subset of **C**-like Language.

Language Scope

The compiler supports the following language constructs:

1. **Data Types:** Int, Float, Bool
2. **Control Structures:** If- else, While loop, For loop
3. **Functions:** Declaration, parameters, and return statement
4. **Operators:** Arithmetic (+, -, *, /), Relational (==, !=, <, >, <=, >=), Logical (&&, ||, !), Assignment
5. **Statements:** Variable declarations, Assignment statements

3. Compiler Architecture & Design

Overall System Design

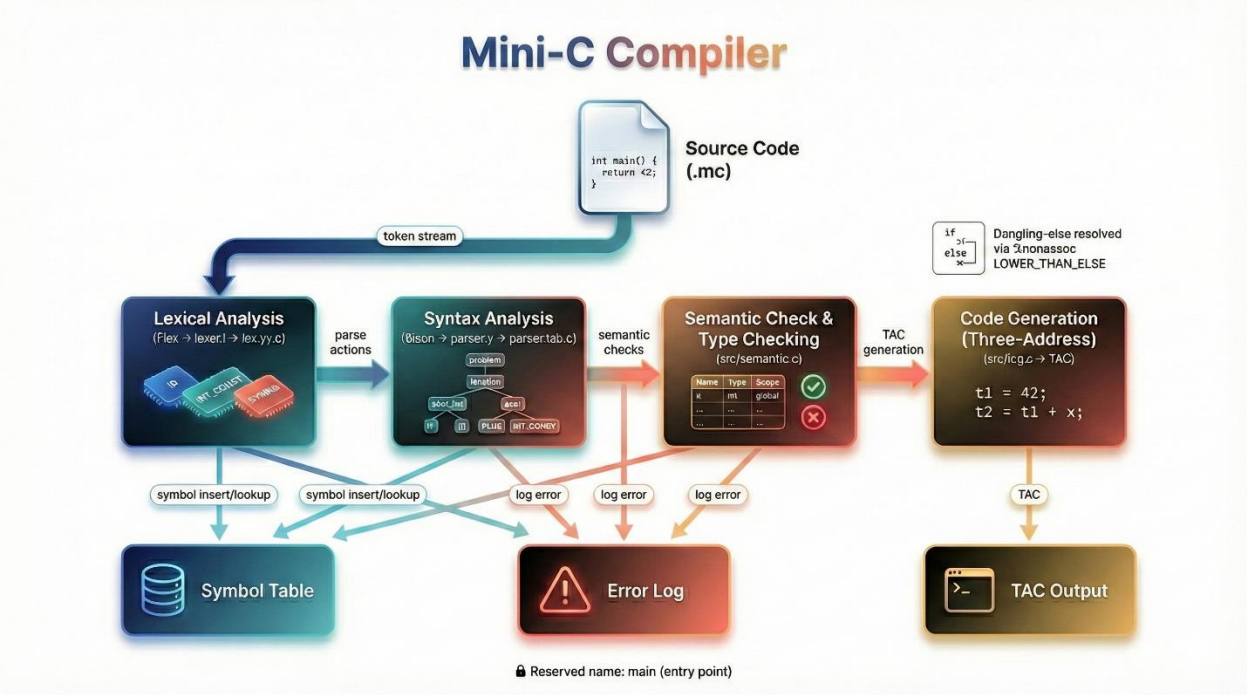

Figure 1:System Design

Modular Component Design

Components	Files	Responsibility
Lexer	lexer.l, lex.yy.c	Tokenize source; track line numbers
Parser	`parser.y`, `parser.tab.c`	LALR (1) grammar; parse actions
Symbol Table	src/symbols.c/h	Scope stack; insertion, lookup; redeclaration detection
Semantic Analyzer	src/semantic.c/h	Type checking; return-type validation; error reporting
Intermediate Code Generation	src/icg.c/h	TAC generation; temporary allocation
Error Handling	src/parser_error_log.h	Error logging infrastructure

4. Implementation Details

Lexical Analysis

The lexical analyzer reads the source program and converts character streams into tokens such as identifiers, keywords, operators, constants, and punctuation.

Token Type	Pattern	Example
Keywords	`if`, `else`, `while`, `for`, `return`, `input`, `output`	`if`, `return`
Type Keywords	`int`, `float`, `bool`, `true`, `false`	`int`
Identifiers	`[a-zA-Z_][a-zA-Z0-9_]`	`x`, `count_var`
Integer Literals	`[0-9]+`	`42`, `0`
Float Literals	`[0-9]+"."[0-9]+`	`3.14`, `2.0`
String Literals	`"[^"\n]*"`	`"Enter x:"`
Operators	+, -, *, /, =, ==, etc.	+, ==, &&
Delimiters	(,), {, }, ;, ,	(,)
Comments	// (line), /* */ (block)	// comment

Figure 2: Token Classes

Syntax Analysis

The parser validates program structure using context-free grammar rules and constructs a parse structure while detecting syntax errors.

Grammar Rules:

```
Program         → decl_list
decl_list       → decl_list decl | decl
decl            → type ID ';' | func_def | error ';'
type            → 'int' | 'float' | 'bool'
func_def        → type ID '(' param_list_opt ')' block
param_list_opt  → param_list | ε
param_list      → param | param_list ',' param
param           → type ID
block           → '{' stmt_list '}'
stmt_list       → stmt_list stmt | ε
stmt            → type ID ';' | assign_stmt | input_stmt | output_stmt
                  | if_stmt | while_stmt | for_stmt | return_stmt  | block_stmt | ';' | error ';'
assign_stmt     → ID '=' expr ';'
input_stmt      → 'input' '(' in_list ')' ';'
```

```
in_list          → ID | in_list ',' ID
output_stmt      → 'output' '(' out_list ')' ';'
out_list         → out_item | out_list ',' out_item
out_item         → STRING | expr
if_stmt          → 'if' '(' expr ')' stmt | 'if' '(' expr ')' stmt 'else' stmt
while_stmt       → 'while' '(' expr ')' stmt
for_stmt         → 'for' '(' for_init ';' expr ';' for_update ')' stmt
for_init         → assign_stmt_no_semi | type ID '=' expr | ε
for_update       → assign_stmt_no_semi | ε
return_stmt      → 'return' expr ';' | 'return' ';'
expr             → INT_CONST | FLOAT_CONST | BOOL_CONST | ID | STRING
                  | expr '+' expr | expr '-' expr | expr '*' expr | expr '/' expr
                 | expr '==' expr | expr '!=' expr | expr '<' expr | expr '<=' expr
                 | expr '>' expr | expr '>=' expr | expr '&&' expr | expr '||' expr
                 | '!' expr | '(' expr ')'
```

Operator Precedence & Associativity:

```
%left LOR          // || (lowest)
%left LAND         // &&
%right LNOT        // !
%left EQ NE LE GE LT GT // relational
%left ADD SUB      // + -
%left MUL DIV      // * / (highest)
%nonassoc LOWER_THAN_ELSE // resolve dangling-else.
```

Semantic Analysis

The semantic analyzer performs type checking, declaration validation, Scope Verification and compatibility checking in expressions & assignment.

Symbol Table Data Structure:

The symbol table stores identifiers along with type information and scope attributes.

```
typedef struct {
  char name[31];          // identifier name
  DataType type;          // TYPE_INT, TYPE_FLOAT, TYPE_BOOL, TYPE_VOID
  SymbolKind kind;        // KIND_VAR, KIND_PARAM, KIND_FUNC
```

```
    int scope_level;         // 0 = global, 1+ = nested blocks
    int initialized;         // 0 = uninitialized, 1 = assigned
    int is_param;            // 1 if function parameter
} Symbol;

typedef struct {
    Symbol symbols[MAX_SYMBOLS];
    int symbol_count;
    int current_scope;
    int scope_top;
    int scope_stack[MAX_SCOPES]; // track start index of each scope
} SymbolTable;
```

Key Operations:

Operation	Purpose
insert_symbol(st, name, type, kind)	Add identifier; return 0 on redeclaration, -1 on reserved-name error
lookup_symbol(st, name)	Find in scope hierarchy
lookup_in_scope(st, name, scope)	Find in specific scope
enter_scope()	Push new scope (function/block)
exit_scope()	Pop scope

Reserved Name Enforcement:

```
if (strcmp(name, "main") == 0 && kind != KIND_FUNC) {
    report_semantic_error(line_no, "Usage of reserved name 'main' as variable is not allowed
");
    return -1;  // Signal error already reported
}
```

Intermediate Code Generation

The compiler generates **Three Address Code (TAC)** to represent program logic in a machine-independent form.

Three Address Code (TAC) Format:

```
t0 = 42          // load constant
t1 = x           // load variable
```

```
t2 = t0 + t1       // binary operation
t3 = "Hello"       // string constant
print t3           // output (placeholder)
read x             // input (placeholder.
```

TAC Generation Functions:

Function	Output Example
gen_const_int(42)	t0 = 42
gen_const_float(3.14)	t1 = 3.14
gen_const_string("msg")	t2 = "msg"
gen_id("x")	t3 = x
gen_binary_op("+")	t4 = t3 + t2
gen_output()	print t4
gen_input("x")	read x

These models used flattened and scaled pixel values, requiring no manual feature engineering beyond normalization.

5. Design Choices

The compiler was implemented using a modular architecture in which each compilation phase is implemented as a separate component and integrated through semantic actions. The design decision to use **Flex for lexical analysis** and **Bison for syntax analysis** allowed automatic generation of scanners and LALR (1) parsers, reducing implementation complexity while preserving correctness. A structured symbol table module was used to manage identifiers, data types, and scope information, ensuring clean separation between parsing and semantic processing.

Three Address Code (TAC) was selected as the intermediate representation because it is machine-independent, simple to generate, and closely resembles low-level program execution. The compiler design prioritizes readability, modularity, and extensibility so that new language constructs can be added with minimal change

6. Test Cases & Results

There are 34 test case files used to test the compiler each result shown in table below:

#	Test File	Result
1	elseif_blocks_correct.mc	Pass
2	elseif_blocks_incorrect.mc	Pass
3	elseif_correct.mc	Pass
4	elseif_incorrect.mc	Pass
5	error_type_mismatch.mc	Pass
6	good_main.mc	Pass
7	if_else_blocks_correct.mc	Pass
8	if_else_blocks_incorrect.mc	Pass
9	if_else_correct.mc	Pass
10	if_else_incorrect.mc	Pass
11	input_correct.mc	Pass
12	input_multiple_correct.mc	Pass
13	input_undeclared.mc	Pass
14	lexical_error.mc	Pass
15	main_as_var.mc	Pass
16	missing_semicolon.mc	Pass
17	output_string_correct.mc	Pass
18	output_string_mixed.mc	Pass
19	output_undeclared.mc	Pass
20	redeclaration.mc	Pass
21	report_good_main.mc	Pass
22	report_io_example.mc	Pass

23	report_main_as_var.mc	Pass
24	test_decl_assign.mc	Pass
25	test_error_type_mismatch.mc	Pass
26	test_error_undeclared.mc	Pass
27	test_expr_arith.mc	Pass
28	test_features.mc	Pass
29	test_function.mc	Pass
30	test_if_else.mc	Pass
31	test_types.mc	Pass
32	test_validation.mc	Pass
33	undeclared_var.mc	Pass
34	x_var.mc	Pass

Some Detail Test Cases

Test 6: good_main.mc

// tests/report_good_main.mc

```
int main() { return 0; }
```

```
=== MiniC Compiler (LALR Parser) ===

Compilation of tests\good_main.mc successful!

=== SYMBOL TABLE ===
Name                 | Type       | Kind       | Scope | Initialized
-------------------------------------------------------------
main                 | int        | FUNC       | 0     | NO
-------------------------------------------------------------


=== Three-Address Code (TAC) ===
  1: t0 = 0
================================
```

Test 6: missing_semicolon.mc

```
int main() {

  int x
  output("Enter x:", x);
return 0;
}
```

```
PS O:\CC LAB\Mini C Compiler> .\bin\minic.exe .\tests\missing_semicolon.mc

=== MiniC Compiler (LALR Parser) ===

Syntax Error at line 3: syntax error

Compilation of .\tests\missing_semicolon.mc failed with 1 error(s)
```

Test 22: report_io_example.mc

```
int main() {   int x;    output("Enter x:", x);   input(x);
  output("You entered:", x);
  return 0;
}
```

```
=== MiniC Compiler (LALR Parser) ===

Compilation of tests\report_io_example.mc successful!

=== SYMBOL TABLE ===
Name                 | Type        | Kind        | Scope | Initialized
-------------------------------------------------------------
main                 | int         | FUNC        | 0     | NO
x                    | int         | VAR         | 1     | NO
-------------------------------------------------------------


=== Three-Address Code (TAC) ===
  1: t0 = "Enter x:"
  2: print t0
  3: t1 = x
  4: print t1
  5: read x
  6: t2 = "You entered:"
  7: print t2
  8: t3 = x
  9: print t3
 10: t4 = 0
```

Test 25: test_error_type_mismatch.mc

```
int main() {
bool flag;
```

```
Flag=42;
 return 0; }
```

```
PS O:\CC LAB\Mini C Compiler> .\bin\minic.exe tests\test_validation.mc

=== MiniC Compiler (LALR Parser) ===

Lexical Error at line 3: Identifier 'this_is_a_very_long_variable_name_that_exceeds_thirty_characters' exceeds maximum length of 30 characters
Syntax Error at line 7: syntax error
Syntax Error at line 8: syntax error
Syntax Error at line 9: syntax error
Syntax Error at line 10: syntax error
Lexical Error at line 10: Float '4.1234567' must have 1-6 decimal digits (found 7)
Syntax Error at line 11: syntax error
Lexical Error at line 11: Unknown symbol .
Syntax Error at line 12: syntax error
Lexical Error at line 12: Float '6.1234567890' must have 1-6 decimal digits (found 10)
Semantic Error: Program must define a global function 'main'
```

Test 32: test_validation.mc

```
int main() {
// Test identifier length validation
int validname;
int this_is_a_very_long_variable_name_that_exceeds_thirty_characters;
int x;
bool flag;
Flag=42;
// Test float decimal digit validation
float a = 3.14;
float b = 2.5;
float c = 1.123456;
float d = 4.1234567;
float e = 5.;
float f = 6.1234567890;
 return 0; }
```

```
=== MiniC Compiler (LALR Parser) ===

Lexical Error at line 3: Identifier 'this_is_a_very_long_variable_name_that_exceeds_thirty_charac
ters' exceeds maximum length of 30 characters                                    Syntax Error at line 7: syntax error
Syntax Error at line 8: syntax error
Syntax Error at line 9: syntax error
Syntax Error at line 10: syntax error
Lexical Error at line 10: Float '4.1234567' must have 1-6 decimal digits (found 7)
Syntax Error at line 11: syntax error
Lexical Error at line 11: Unknown symbol .
Syntax Error at line 12: syntax error
Lexical Error at line 12: Float '6.1234567890' must have 1-6 decimal digits (found 10)
Semantic Error: Program must define a global function 'main'

Compilation of tests\test_validation.mc failed with 8 error(s)
```

7. Error Recovery Methods

This compiler uses panic mode recovery error technique for errors the table below show error message references.

Error Type	*Format*
Lexical	Lexical Error at line N: Unknown symbol '@'
Syntax	Syntax Error at line N: syntax error
Undeclared	Semantic Error at line N: Undeclared variable 'X'
Type Mismatch	Semantic Error at line N: Type mismatch in assignment (expected int, got bool)
Redeclaration	Semantic Error at line N: Redeclaration of 'X'
Reserved Name	Semantic Error at line N: Usage of reserved name 'main' as variable is not allowed
Missing main	Semantic Error: Program must define a global function 'main'

8. Lesson Learned & Insights

Compiler Theory Insights

1. **Grammar Clarity is Critical:** Dangling-else ambiguity took significant debugging. A clear grammar specification avoids such issues.
2. **Symbol Table is Central:** The symbol table connects all compiler phases. Proper design (scope stack, lookup semantics) makes semantic analysis straightforward.
3. **Error Messages Are User-Facing:** Detailed line-number errors and clear messages greatly improve usability compared to generic “parse error” messages.
4. **One-Pass vs. Two-Pass Trade-Off:** This compiler is one-pass (semantic actions during parsing). Two-pass would be cleaner but slower.
5. **Memory Safety Matters:** String allocations in lexer/parser require careful free() calls. Missing even one path causes leaks or crashes.

Project Execution Insights

1. **Modular boundaries:** Each file (lexer.l, parser.y, symbols.c, etc.) can be developed independently with clear interfaces.
2. **Automated testing:** Test suite catches regressions immediately; easy to validate changes.
3. **Documentation-as-you-go:** Grammar rules and error handling should be documented during coding, not afterwork could incorporate data augmentation and more advanced regularization to push performance closer to human-level accuracy.

9. Conclusion

This project successfully implemented a mini-compiler integrating lexical analysis, syntax analysis, semantic checking, symbol table management, and intermediate code generation. It provided practical understanding of compiler phases and their system-level interaction. The compiler correctly handled valid programs and identified semantic and structural errors with meaningful messages. Overall, the project strengthened our conceptual and implementation skills in compiler construction and systems programming.